



University of Glasgow | School of
Computing Science

DayOfWeek DayOfMonth Month 2XXX

XX.XX am/pm – XX.XX am/pm

(Duration: 1 hour)

EXAMINATION FOR

DEGREES OF MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

BD4 Exam Paper

Exam Rubric: Answer All Questions

INSTRUCTIONS TO INVIGILATORS

Please collect all exam question papers and return to School together with
exam answer scripts

Questions:

1. What are the main drawbacks (as discussed in the class) of employing relational Database Systems for managing big data? [10 points]
2. Explain the global and local (per node) metadata maintained by HDFS per HDFS file? [10 points]
3. What are the main entities in the MapReduce2 framework? Briefly, present their roles. [10 points]
4. In the class we discussed several master-slave systems, (e.g., HDFS, MapReduce, HBase) all of which desired recoverability. Explain the principles for recoverability – specifically referring to Write Ahead Logs (WAL), journals, checkpoints, and backup (standby) nodes. [10 points]
5. What are the key design/architecture differences and similarities between MapReduce1 and MapReduce2 and their implications on system performance, scalability, and fault tolerance? [10 points]
6. Explain the procedure followed by HBase whenever a data item is requested (i.e., how it is located). [10 points]

MARKING SCHEME

Assigned marks will be equal to the proportion of the answer each student answer provides. That is, an answer covering 70% of the answer below, will receive 70% of the allotted mark.

ANSWERS

Q1. Drawbacks of R-DBMSs for big data.

One cannot typically rely on employing multiple R-DBMSs for scaling, as these systems were built for different applications and come bundled with design decisions not appropriate for big data applications. Specifically, there are two main drawbacks:

a. **Normalisation.** R-DBMSs depend on normalization processes, which yield a fairly large *number of tables* with *foreign key* relations between them. Big data applications typically need *access to information (attributes) spread across several tables*. This requires expensive *multi-way join operations*. Join operations are notoriously *expensive* and resource hungry to execute. With *big tables*, this would be intolerable for many applications.

b. **Consistency.** R-DBMSs typically enforce *strong consistency (transactional)* semantics for groups of operations, defining transactions. Transactional semantics guard both against *concurrent accesses* of different-user transactions on the same data items and against *failures* which may lead to only parts of a transaction's operations being successful. For the former, expensive distributed concurrency control methods are required. For the latter, expensive distributed commit protocols are required. In many big data applications such *grouping of an application's logic into transactions is not required* and hence the cost associated with transactional semantics can be avoided.

Q2. HDFS global and local metadata per file:

a. **Global** metadata. Maintained by the *NameNode*. It consists primarily of the *access restrictions* for each file (so that it can honour or deny access to datanodes and their files), the *location information* (datanodes) of *file blocks* and their *replicas*, and the *state of the namespace* (the complete filename tree of all files/directories in existence).

b. **Local** metadata. Maintained by each *DataNode* for each file it stores. Primarily it consists of the *file sizes* (communicated to NN for global space management) and the *file checksums* (critical for recovery from failures).

Q3. Hadoop MR2 entities and roles.

Entities and roles:

- The *Client*: *submits* MR jobs
- The *Resource Manager*: *Allocates* cluster resources (in terms of *containers*) and schedules execution of MR jobs on these resources.
- *Node Managers*: run at *each cluster node*. *Launch* and *monitor* the node's containers.
- Application master (AM): a per MR job daemon that
 - Manages the job, coordinates tasks of job.
 - Negotiates with RM for resources
- HDFS for MR job I/O

Q4. Recoverability principles and tools

Recoverability is concerned with not losing crucial data for the system due to failures (node crashes, etc.). Principally, there are 3 resources to use for recovery: the **image**, the **journal**, and the **checkpoint**.

- The **image** is an *in-memory* data resource holding the key state of the system (for example, for the HDFS, this is basically the filename space). This is maintained in main memory in order to offer *efficient access*. But, main memory is *volatile*, so stable storage for this state (e.g., on disks) is needed if system is to survive failures.
- Periodically, to facilitate recovery, the contents of the in-memory image is dumped to stable storage (disk) for *durability*. The **checkpoint** is this resource.
- In between successive checkpoints of the image, all new operations mutating the state are recorded in a log in stable storage (disk), called the **journal**. This log is *write-ahead*, meaning that first operations are saved there and then performed on the state in main-memory.

With these resources at hand, upon recovery, the system re-initializes its state to the last saved checkpoint and then replays the operations in the journal on this image.

Additionally, a **backup node** may exist, which also stores replicas of the image and/or the journal and/or the checkpoints to further increase recoverability.

Q5. What are the key design/architecture differences and similarities between MapReduce1 and MapReduce2 and implications?

In MR1 the main entities are: client, job tracker (JT), task trackers (TT), and the HDFS. MapReduce is the only service supported in MR1, and the JT/TTs take on the tasks of both managing computing resources *and* job scheduling/execution. In MR1 the client submits a job (code, libraries, scripts, etc.) to the JT, which then assigns parts of the job to mapper/reducer slots on the TTs; TTs are responsible for executing these tasks and for reporting progress back to the JT.

On the other hand, MR2 has decoupled the resource management from the MapReduce job execution. In MR2, YARN (Yet Another Resource Negotiator) is responsible for allocating memory and cpu cores to applications, only one of which is MapReduce. Thus, in MR2 the main entities are: client, RM (the Resource Manager – responsible for scheduling and resource management at a cluster level), the Node Manager (NM – responsible for scheduling and resource management at a node level), the Application Master (AM), and HDFS. Obviously the client entity and HDFS are common in both. However, in MR2 the client first requests a *container* (i.e., set of resources) from the RM to execute an AM, which takes on the role of JT in MR1. The RM schedules this container on one of the NMs and executes the AM, which then requests containers for individual subtasks from the RM. These containers are the rough equivalent of TTs; they are first assigned to NMs, taking into account data locality, then a YarnChild process is launched in them (the equivalent of a TT), then the AM schedules mapper/reducer subtasks on each of these YarnChild's, which execute them and report their progress back to the AM. Thus, in MR2 the RM and NM are not directly related to MapReduce jobs; they store no info related to the actual jobs, other than info for the containers. It's the job of the AM and YarnChild processes to look after MR jobs.

Other than that, the rest of the job execution (I/O, M/R phases, progress reporting/monitoring, etc.) is the same across both MR1 and MR2, and from the point of view of applications/jobs MR2 is fully backwards compatible with MR1.

Q6. Explain the procedure followed by HBase whenever a data item is requested (i.e., how it is located).

In HBase the main architectural components are the client, the HBase Master (HM), a number of RegionServers (RS), and Zookeeper (ZK – a separate distributed service, possibly running on nodes other than those hosting the HBase components, and operating on a consensus basis). HBase manages data stored on HDFS, so one can think of the HDFS components (NameNode – NN, DataNodes – DN) as underlying parts of the HBase architecture. The HM is responsible for assigning regions to RSs and for monitoring their health. ZK, in turn, is used to maintain shared state in the cluster, including which RS is alive or not, and to serve as a queue for RSs' heartbeats being sent to the HM.

There is a special HBase table, coined the META table, storing a mapping of regions to RSs. This table is managed by a single RS and resides on a single server (although this will change in the near future). When a client requests access to a data item, it first contacts the ZK ensemble to get the (ip, port) info of the RS storing the META table. This location information used to be stored in another special table (called -ROOT-) but in recent versions of HBase it has been superseded and ZK is used to store this information. The client then queries this table to get the location (RS info) of the region that includes the rowkey it is trying to access. Finally, it contacts the RS storing this region. This information is cached at the client so that subsequent accesses to the same region won't have to query the META table, unless there is a miss (i.e., the region has been reassigned to another region server).

Last, the RSs manage regions in the sense that they maintain mappings of regions to HDFS blocks; there is no guarantee that a region managed by a given RS will be stored on a DN collocated with the RS, or that RSs and DNs are collocated at all in the first place (although Hadoop/HBase strive to maintain data locality). Whenever our client above is accessing a row in a table, the RS responsible for the corresponding region is acting as a mediator between the client and HDFS, by reading the data off of the DN storing the region block, caching it in its block cache, and returning the row data to the client. In order to locate said DN, the RS is acting as a client to HDFS; i.e., it accesses the NN to get a mapping of the region's blocks to DNs storing replicas of them, and then accesses a DN directly.